



MASTER SOLUTION — UNIT I: INTRODUCTION TO FORMAL LANGUAGES AND FINITE AUTOMATA

Subject: Theory of Computation

Branch: Computer Engineering

Pattern: 2024

Sem: V

Total Questions: 18

Syllabus: Basic Concepts: Finite and infinite set, Symbols, Strings: Empty String, Substring of a string, Concatenation of strings, Language: Formal Language Definition, Finite representation of languages, Operations on languages: Union, Concatenation, Kleene star and Kleene plus, Concept of Basic Machine. FA without output: Finite State Machines (FSM), Deterministic and Nondeterministic FA (DFA & NFA), epsilon NFA, Conversion of NFA with epsilon moves to NFA, Conversion of NFA to DFA, and Conversion of NFA with epsilon moves to DFA, Minimization of DFAs. FA with output: Moore and Mealy machines - Definition, Construction, Inter-Conversion.

Q1 Explain the basic concepts of sets, symbols, and strings used in formal language theory. Define empty string, substring and concatenation of strings with suitable examples. [8M]

Ans

Formal language theory begins with a handful of primitive building blocks — sets, symbols, alphabets and strings — on top of which grammars, automata and languages are defined. Understanding these primitives precisely is essential because every later construction, from a DFA to a Turing Machine, is stated in terms of them.

Set: A set is a well-defined, unordered collection of distinct objects called elements. A set can be finite, having a countable fixed number of elements such as $A = \{0, 1, 2\}$, or infinite, having an unbounded number of elements such as the set of natural numbers $N = \{0, 1, 2, 3, \dots\}$. In automata theory the alphabet itself is always taken to be a finite non-empty set, while the language built over it may be finite or infinite.

Finite Set: A finite set contains a specific, countable number of elements and this count is called its cardinality. Example: the set of vowels $\{a, e, i, o, u\}$ has cardinality 5. Finite sets are used to represent alphabets, sets of states of an automaton, and sets of final states.

Infinite Set: An infinite set has no last element and its elements cannot be exhausted by counting. Example: the set of all strings over $\{0,1\}$, denoted $\{0,1\}^*$, is infinite because strings of arbitrarily large length can always be formed. Most interesting languages studied in Theory of Computation are infinite sets of strings, which is precisely why finite machines are required to represent them compactly.

Symbol: A symbol is an abstract, atomic entity that cannot be broken down further for the purposes of the formal system — for example 'a', 'b', '0', '1'. Symbols are the smallest units of an alphabet and act like the alphabet letters of ordinary language.

Alphabet (Σ): An alphabet is a finite, non-empty set of symbols. For example $\Sigma = \{0, 1\}$ is the binary alphabet and $\Sigma = \{a, b\}$ is used commonly in automata examples. Every string and every language discussed in the course is built strictly from symbols of some fixed alphabet Σ .

String: A string (also called a word) is a finite sequence of symbols drawn from an alphabet Σ , written by juxtaposing the symbols one after another with no separators. For example, over $\Sigma = \{a, b\}$, 'abba' and 'aab' are valid strings. The length of a string w , written $|w|$, is the total number of symbol occurrences in it; for instance $|abba| = 4$.

Empty String (ϵ): The empty string, denoted ϵ (or sometimes λ), is the unique string that contains no symbols at all. Its length is 0, i.e. $|\epsilon| = 0$. It behaves as the identity element for the concatenation operation, meaning that for every string w , $w \cdot \epsilon = \epsilon \cdot w = w$. Every alphabet Σ , no matter how large, always contains exactly one empty string in Σ^* .

Substring: A substring of a string w is any contiguous block of symbols that occurs within w , obtained by deleting a prefix and a suffix (possibly empty) from w . For example, if $w = 'abcde'$, then 'bcd', 'ab', 'cde', and even w itself and ϵ are all substrings of w . A substring must preserve the original left-to-right order and contiguity of the symbols; 'ace' is not a substring of 'abcde' because the symbols are not contiguous in w , even though they occur in the correct relative order — such non-contiguous selections are instead called subsequences.

Concatenation of Strings: Concatenation is the fundamental binary operation on strings, which joins two strings end to end to produce a new, longer string. If x and y are strings, their concatenation is written xy or $x \cdot y$, and it is formed by appending the symbols of y immediately after the symbols of x . For example, if $x = 'car'$ and $y = 'pet'$, then $xy = 'carpet'$. Concatenation is associative, i.e. $(xy)z = x(yz)$, but it is not commutative in general since xy need not equal yx .

Worked Example: Let $\Sigma = \{a, b\}$, $x = 'ab'$ and $y = 'ba'$. Then $|x| = 2$, $|y| = 2$, the concatenation $xy = 'abba'$ with $|xy| = 4$, and $yx = 'baab'$, showing $xy \neq yx$ in general. The strings 'a', 'b', 'ab', 'bb' and ϵ are all substrings of 'abba', while 'ba' occurring at position 2-3 is also a valid substring.

Concept	Meaning	Example
Set	Unordered collection of distinct objects	$\{0,1,2\}$
Symbol	Smallest atomic unit of an alphabet	'a', '0'
Alphabet Σ	Finite non-empty set of symbols	$\{a,b\}$
String	Finite sequence of symbols from Σ	'abba'
Empty String ϵ	String of length 0	ϵ
Substring	Contiguous block within a string	'bc' in 'abcd'
Concatenation	Joining two strings end to end	'ab'.'ba' = 'abba'

These primitives together give a precise mathematical vocabulary — every regular expression, grammar production, and machine transition studied further in the subject is ultimately expressed as an operation on strings drawn from a fixed alphabet.

Q2	What is a formal language? Explain the finite representation of languages and the operations of union, concatenation, Kleene star and Kleene plus on languages, giving suitable examples for each. [9M]
Ans •	A formal language provides the mathematical framework needed to describe precisely which strings are 'valid' according to some rule, independent of any natural-language meaning. This notion underlies programming language syntax, protocol specifications, and pattern matching. Formal Language — Definition: Given an alphabet Σ, let Σ^* denote the set of all possible strings that can be formed using symbols from Σ, including the empty string ϵ. A formal language L is defined simply as any subset of Σ^*, i.e. $L \subseteq \Sigma^*$. A language is therefore nothing but a chosen collection of strings, and it may be finite (containing a fixed number of strings) or infinite (containing infinitely many strings). Finite Representation of Languages: Since a language can contain infinitely many strings, it cannot always be listed out explicitly. Theory of Computation therefore studies finite mechanisms that can represent an infinite language compactly. Three standard finite representations are used throughout the subject. • Set-builder notation: A language is described using a defining property, such as $L = \{a^n b^n \mid n \geq 1\}$, which stands for the infinite set $\{ab, aabb, aaabbb, \dots\}$ without listing every string. • Regular expressions / grammars: A finite set of symbols and operators (or finite production rules of a grammar) can generate an infinite language, e.g. the regular expression a^*b^* finitely represents every string of the form $a^i b^j$ for $i, j \geq 0$. • Automata (machines): A finite automaton with a bounded number of states can accept an infinite language by looping over repeated input symbols, e.g. a two-state DFA accepts the infinite language of all strings with an even number of 1's. Operations on Languages: Because languages are simply sets of strings, standard set operations as well as string-specific operations can be defined on them. Let L_1 and L_2 be two languages over a common alphabet Σ. • Union ($L_1 \cup L_2$): The union contains every string that belongs to L_1 or to L_2 or to both. Formally, $L_1 \cup L_2 = \{w \mid w \in L_1 \text{ or } w \in L_2\}$. Example: if $L_1 = \{a, ab\}$ and $L_2 = \{b, ba\}$, then $L_1 \cup L_2 = \{a, ab, b, ba\}$. • Concatenation ($L_1 L_2$): The concatenation of two languages contains every string formed by taking one string from L_1 and appending one string from L_2 immediately after it. Formally, $L_1 L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}$. Example: if $L_1 = \{a, aa\}$ and $L_2 = \{b, bb\}$, then $L_1 L_2 = \{ab, abb, aab, aabb\}$. • Kleene Star ($L^*$): The Kleene star of a language L is the set formed by concatenating zero or more strings of L together in any order, including the empty string ϵ (obtained by concatenating zero strings). Formally, $L^* = L^0 \cup L^1 \cup L^2 \cup \dots$. Example: if $L = \{a, b\}$, then $L^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$ — that is, all possible strings over $\{a, b\}$ of any length including zero. • Kleene Plus (L^+): The Kleene plus is identical to the Kleene star except that ϵ is excluded unless it is already present in L itself; it requires concatenating one or more strings of L. Formally, $L^+ =$

$L^1 \cup L^2 \cup L^3 \dots = L^* - \{\epsilon\}$ (when $\epsilon \notin L$). Example: if $L = \{a, b\}$, then $L^+ = \{a, b, aa, ab, ba, bb, aaa, \dots\}$, which excludes only the bare empty string ϵ that L^* would have included.

Operation	Formal Definition	Example ($L1=\{a\}, L2=\{b\}$)
Union	$L1 \cup L2 = \{w \mid w \in L1 \text{ or } w \in L2\}$	$\{a, b\}$
Concatenation	$L1L2 = \{xy \mid x \in L1, y \in L2\}$	$\{ab\}$
Kleene Star	$L^* = \bigcup L^n, n \geq 0$ (includes ϵ)	$L1^* = \{\epsilon, a, aa, \dots\}$
Kleene Plus	$L^+ = \bigcup L^n, n \geq 1$ (excludes ϵ unless in L)	$L1^+ = \{a, aa, aaa, \dots\}$
Intersection	$L1 \cap L2 = \{w \mid w \in L1 \text{ and } w \in L2\}$	$\{\}$ (disjoint here)
Complement	$L' = \Sigma^* - L$	all strings except 'a'

These closure operations are important because regular languages are closed under all of union, concatenation, Kleene star and complement — a fact used repeatedly to prove that a newly built language is regular, simply by expressing it in terms of simpler regular languages combined with these operations.

Q3

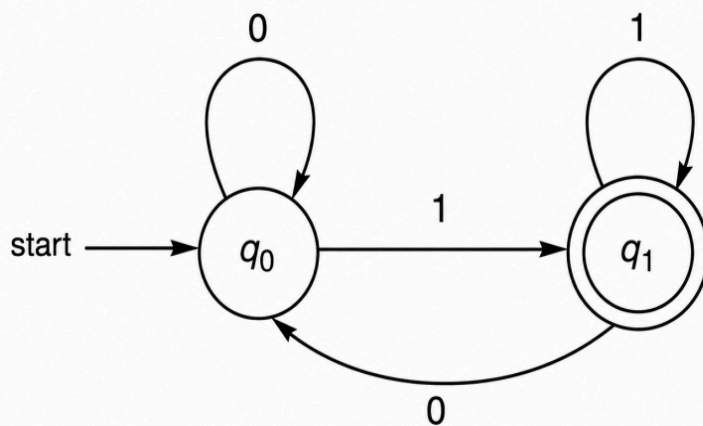
Explain the concept of a 'Basic Machine' and give an informal picture of a Finite Automaton (FA), describing its essential components. [6M]

Ans

- Before defining a Finite Automaton formally, it helps to visualise it informally as a 'basic machine' — the simplest possible abstract computing device that reads an input and decides whether to accept or reject it.

Concept of Basic Machine: A basic machine, in the context of automata theory, is an idealised device consisting of three parts: an input tape divided into cells holding one input symbol each, a read head that scans the tape strictly from left to right one symbol at a time, and a finite control unit that keeps track of the current 'state' of the machine. At every step the finite control reads the symbol under the head, consults its transition rule, moves to a new state, and advances the head one cell to the right. This basic machine has no auxiliary memory beyond its own finite set of states, which is exactly what distinguishes a finite automaton from more powerful models such as a pushdown automaton (which adds a stack) or a Turing Machine (which adds an infinite read/write tape).

Informal Picture of FA: A Finite Automaton can be pictured as a directed graph called a transition diagram or state diagram. Circles represent the states of the machine, and directed arrows (edges) labelled with input symbols represent the transitions between states. One special state is marked with an incoming unlabelled arrow to show it is the start state, and states that represent successful acceptance are drawn with a double circle to mark them as final (accepting) states.

Fig — Informal Picture of a Finite Automaton**Fig — Informal Picture of a Finite Automaton**

Working of the Basic Machine: The machine begins in the start state with the head positioned at the leftmost symbol of the input string. On reading each symbol it follows the uniquely (for a DFA) or possibly multiply (for an NFA) defined transition to move to the next state. Once every symbol of the input has been consumed, the machine's final resting state decides the outcome: if that state is one of the designated final states, the input string is said to be accepted, and otherwise it is rejected.

Language Accepted by FA: The set of all strings that drive the machine from the start state to some final state is called the language accepted by that FA. Since the number of states is always finite, but the number of possible strings over an alphabet is infinite, this basic machine gives a finite, compact way of representing what can be an infinite language — which is exactly the practical motivation for using automata to specify languages instead of listing their strings individually.

This informal, graphical view of the basic machine is later made fully rigorous by defining a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, which is the formal definition of a Deterministic or Non-deterministic Finite Automaton.

Q4

Define Finite State Machine (FSM). Explain the formal definition of a Deterministic Finite Automaton (DFA) with a suitable example. [6M]

Ans

A Finite State Machine (FSM) is an abstract mathematical model of computation that consists of a finite number of states, transitions between those states triggered by input symbols, and designated start and final states, used to represent systems whose behaviour can be described purely in terms of a limited number of internal configurations.

Finite State Machine (FSM): An FSM is a computational model in which the machine, at any moment, exists in exactly one of a finite number of states, and moves from one state to another in response to some external input symbol, according to a transition function. FSMs are broadly divided

into FA without output (DFA, NFA), which simply accept or reject an input string, and FA with output (Moore and Mealy machines), which produce an output associated with the transitions or states.

Deterministic Finite Automaton (DFA) — Formal Definition: A DFA is formally defined as a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ where each component has a precise meaning.

- **Q:** a finite non-empty set of states of the machine.
- **Σ :** a finite non-empty set of input symbols, called the input alphabet.
- **δ :** the transition function $\delta: Q \times \Sigma \rightarrow Q$, which takes exactly one current state and one input symbol and returns exactly one next state — this single-valued, total nature of δ is what makes the automaton 'deterministic'.
- **q_0 :** the start state (also called initial state), where $q_0 \in Q$, at which the machine begins processing every input string.
- **F:** the set of final (accepting) states, where $F \subseteq Q$; a string is accepted if the machine ends in a state belonging to F after consuming the entire string.

Formula Name: Extended Transition Function

Expression: $\delta(q, wa) = \delta(\delta(q, w), a)$

Where:

δ = extended transition function over strings, not just single symbols

w = input string, a = single input symbol

Base case: $\delta(q, \epsilon) = q$

Condition: Used to formally determine the state reached after processing a full string, one symbol at a time

Language Accepted by a DFA: The language accepted by DFA M, denoted $L(M)$, is defined as $L(M) = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}$ — the set of all strings that, starting from q_0 , drive the machine into some accepting state.

Illustrative Example: Consider a DFA that accepts all binary strings ending in '1'. Here $Q = \{q_0, q_1\}$, $\Sigma = \{0,1\}$, q_0 is the start state, $F = \{q_1\}$. The transition function is $\delta(q_0,0)=q_0$, $\delta(q_0,1)=q_1$, $\delta(q_1,0)=q_0$, $\delta(q_1,1)=q_1$. For input '101', the machine moves $q_0 \rightarrow(1) q_1 \rightarrow(0) q_0 \rightarrow(1) q_1$, ending in $q_1 \in F$, so '101' is accepted. For input '100', the machine moves $q_0 \rightarrow(1) q_1 \rightarrow(0) q_0 \rightarrow(0) q_0$, ending in $q_0 \notin F$, so '100' is rejected.

Fig — DFA accepting strings ending in 1

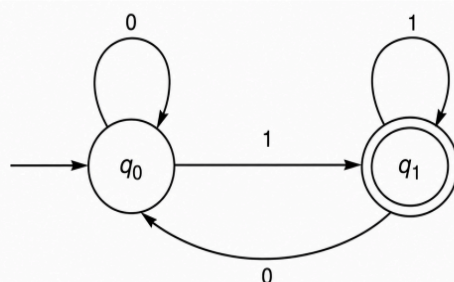


Fig — DFA accepting strings ending in 1

Because δ maps every (state, symbol) pair to exactly one next state, a DFA never has to guess or branch — at every step of processing there is exactly one path the computation can follow, which makes DFAs straightforward to simulate directly in software with a simple lookup table.

Q5

Differentiate between Deterministic Finite Automaton (DFA) and Non-deterministic Finite Automaton (NFA) with suitable examples. [6M]

Ans

DFA and NFA are the two basic variants of finite automata without output. Although they differ in their transition behaviour, a fundamental theorem of automata theory proves that both variants have exactly the same expressive power — every language accepted by an NFA can also be accepted by some DFA.

Deterministic Finite Automaton (DFA): In a DFA the transition function $\delta: Q \times \Sigma \rightarrow Q$ maps every (state, input symbol) pair to exactly one unique next state. There is no ambiguity, no choice, and no missing transition — a DFA transition table is completely filled in for every state and every symbol of the alphabet.

Non-deterministic Finite Automaton (NFA): In an NFA the transition function $\delta: Q \times \Sigma \rightarrow 2^Q$ maps a (state, input symbol) pair to a set of possible next states (possibly the empty set, possibly more than one state). At each step the machine can be thought of as exploring several possible paths of computation simultaneously; a string is accepted if at least one of these many possible paths ends in a final state after the whole string is consumed.

Sr. No.	Parameter	DFA	NFA
1	Transition Function	$\delta: Q \times \Sigma \rightarrow Q$ (single next state)	$\delta: Q \times \Sigma \rightarrow 2^Q$ (set of next states)
2	Number of paths	Exactly one path for every input string	Possibly many paths for the same input string
3	Missing transitions	Every state-symbol pair must be defined (dead state used if needed)	Transitions may be left undefined for a state-symbol pair
4	ϵ -moves	Not permitted in standard DFA	Permitted in ϵ -NFA variant
5	Acceptance condition	Accept if the single computation path ends in a final state	Accept if at least one computation path ends in a final state
6	Memory / implementation	Simple, requires only current state to be tracked	Requires tracking a set of current states, or conversion to DFA
7	Construction ease	Often harder to design directly for complex languages	Usually easier and more compact to design for complex languages
8	Equivalent power	Accepts exactly the regular languages	Accepts exactly the regular languages (same power as DFA)

Illustrative Example: Consider the language $L = \{\text{strings over } \{0,1\} \text{ that contain '01' as a substring}\}$. An NFA for this language needs only three states: q_0 (start), q_1 (reached after seeing 0), q_2 (final,

reached after seeing 1 following that 0), with $\delta(q_0, 0) = \{q_0, q_1\}$ showing two choices on symbol 0. Converting this NFA to an equivalent DFA using the subset construction method typically requires more states, because each DFA state must represent a distinct subset of NFA states that could be simultaneously active.

In practice, NFAs are preferred for design because non-determinism keeps the machine smaller and closer to the informal statement of the language, while DFAs are preferred for implementation and minimization because their deterministic behaviour is directly executable without any backtracking or parallel simulation.

Q6 What is an epsilon-NFA (ϵ -NFA)? Explain the concept of epsilon-closure with a suitable example. [5M]

Ans . An ϵ -NFA extends the ordinary NFA model by permitting the machine to change state spontaneously, without consuming any input symbol at all — such a transition is called an epsilon move.

Epsilon-NFA — Definition: An ϵ -NFA is formally a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$, identical in structure to an NFA, except that the transition function is now $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$. This means that in addition to transitions triggered by an input symbol, the machine may also have transitions labelled ϵ that move it from one state to another without reading anything from the input tape. Epsilon moves are extremely useful when combining smaller automata into a larger one, for example when converting a regular expression into an automaton using Thompson's construction, since sub-machines can simply be 'glued together' using ϵ -transitions.

Epsilon-Closure (ϵ -closure): The epsilon-closure of a state q , written $\text{ECLOSE}(q)$ or $\epsilon\text{-closure}(q)$, is defined as the set of all states that can be reached from q by following zero or more ϵ -transitions, without consuming any input symbol. By definition, $\epsilon\text{-closure}(q)$ always includes q itself, since zero ϵ -moves are allowed. The ϵ -closure of a set of states S is simply the union of the ϵ -closures of each individual state in S .

Formula Name: Epsilon-Closure

Expression: $\epsilon\text{-closure}(q) = \{q\} \cup \{p \mid q \rightarrow \epsilon \rightarrow \dots \rightarrow \epsilon \rightarrow p\}$

Where:

q = a given state of the ϵ -NFA

p = any state reachable from q using only ϵ -transitions

Condition: Computed by a reflexive-transitive closure over the ϵ -transition relation

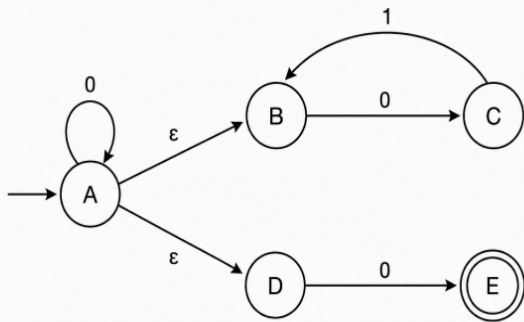
Worked Example: Consider an ϵ -NFA with states $\{A, B, C, D, E\}$ where A has an ϵ -transition to B and a self-loop on 0 , and B has an ϵ -transition to D . Then $\epsilon\text{-closure}(A) = \{A, B, D\}$, because starting at A , one ϵ -move reaches B , and a further ϵ -move from B reaches D ; no ϵ -transition leaves D so the closure stops there. Similarly $\epsilon\text{-closure}(B) = \{B, D\}$ and $\epsilon\text{-closure}(C) = \{C\}$ if C has no outgoing ϵ -moves.

Role in Conversion Algorithms: The ϵ -closure computation is the essential first step used both while converting an ϵ -NFA to an ordinary NFA and while converting an ϵ -NFA directly to a DFA. The start state of the resulting NFA/DFA is taken as $\epsilon\text{-closure}(q_0)$, and for every input symbol a , the next set of

states is computed as the ϵ -closure of the union of $\delta(p, a)$ over every state p currently in the active set — ensuring that any spontaneous transitions available after reading a symbol are also accounted for before the next input symbol is processed.

Convert the given NFA with epsilon moves into an equivalent NFA, and further convert that NFA into its equivalent DFA. Show all intermediate epsilon-closure computations. [10M]

Q7



Ans

The conversion of an ϵ -NFA to a DFA is carried out in two logical stages: first the ϵ -moves are eliminated to obtain an equivalent ordinary NFA, and then the standard subset construction method is applied on that NFA to obtain the DFA. Both stages are shown below using the epsilon-closure concept as the core tool.

Step 1 — Compute Epsilon-Closures of all states: The ϵ -closure of every state is computed first, since it will be needed repeatedly in both stages of the conversion.

State	ϵ -closure	Reasoning
A	{A,B,D}	A reaches B and D directly via ϵ ; B and D have no further outgoing ϵ -moves
B	{B}	B has no outgoing ϵ -transition
C	{C}	C has no outgoing ϵ -transition
D	{D}	D has no outgoing ϵ -transition
E	{E}	E has no outgoing ϵ -transition

Step 2 — NFA without epsilon (elimination of ϵ -moves): For each state q and each real input symbol a , the new transition is computed as $\delta'(q, a) = \epsilon\text{-closure}(\cup \delta(p, a))$ for every p in $\epsilon\text{-closure}(q)$. The new start state remains A, and a state becomes final in the new NFA if its ϵ -closure contains any original final state (here E).

State	On 0	On 1	Final?
A	{A,B,C,E} (A→A via 0; B→C via 0; D→E via 0, all within closure {A,B,D})	{ } (no symbol-1 move directly out of A,B,D)	No

B	{C}	{}	No
C	{}	{B}	No
D	{E}	{}	No
E	{}	{}	Yes (final)

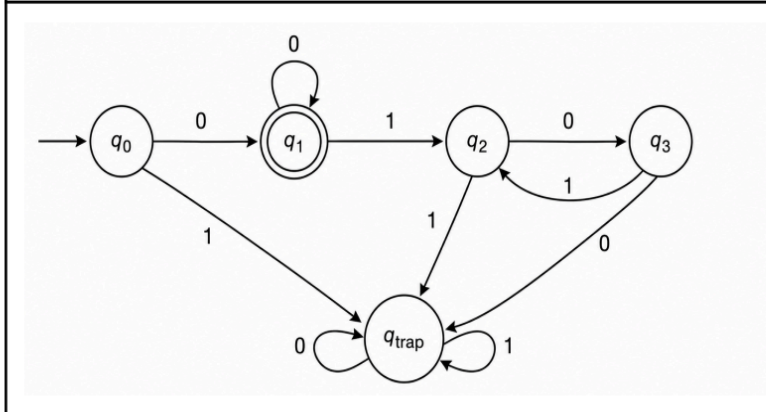
The set $\{A,B,C,E\}$ obtained for A on input 0 shows how eliminating ϵ -moves can merge several original destinations into a single non-deterministic transition; this NFA is now free of ϵ -moves but is still non-deterministic since A goes to more than one state conceptually and needs the subset method next.

Step 3 — Subset Construction (NFA to DFA): Starting from the start state of the NFA, every reachable subset of NFA states is treated as a single DFA state, until no new subsets are generated.

DFA State	On 0	On 1	Final?
$q_0 = \{A\}$	$\{A,B,C,E\} = q_1$	$\emptyset = \text{qtrap}$	No
$q_1 = \{A,B,C,E\}$	$\{A,B,C,E\} = q_1$	$\{B\} = q_2$	Yes
$q_2 = \{B\}$	$\{C\} = q_3$	$\emptyset = \text{qtrap}$	No
$q_3 = \{C\}$	$\emptyset = \text{qtrap}$	$\{B\} = q_2$	No
$\text{qtrap} = \emptyset$	qtrap	qtrap	No

Final Result: The resulting DFA has five states $\{q_0, q_1, q_2, q_3, \text{qtrap}\}$, start state q_0 , and the single accepting state q_1 (since it

Fig — Equivalent DFA obtained after full conversion

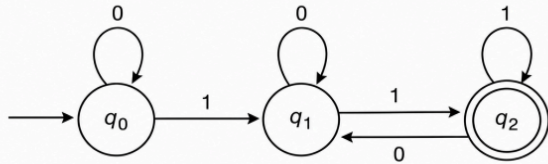


is the only subset that contains original final state E). qtrap is the dead/trap state absorbing all further transitions once a string can no longer reach an accepting configuration.

This two-stage method — first removing ϵ -moves using closures, then applying subset construction — is the standard textbook approach and generalises directly to any ϵ -NFA regardless of the number of states or the complexity of its transition diagram.

Q8

Convert the following NFA into its equivalent DFA using the subset construction method. [8M]



Ans

Subset construction (also called the powerset construction) systematically converts a non-deterministic automaton into a deterministic one by treating every reachable set of NFA states as a single state of the new DFA. It is guaranteed to terminate because the number of distinct subsets of a finite state set Q is itself finite, bounded above by $2^{|Q|}$.

Algorithm — Subset Construction: The steps below are followed mechanically to build the transition table of the DFA from the NFA.

- **Step 1:** Start with the DFA start state as the set $\{q_0\}$, containing only the NFA's start state.
- **Step 2:** For the current unmarked DFA state (a subset of NFA states) and each symbol a of the alphabet, compute the union of $\delta(p, a)$ for every NFA state p in that subset; this union becomes a new DFA state.
- **Step 3:** Mark the current DFA state as processed, and repeat Step 2 for every newly discovered, unmarked subset until no new subsets are produced.
- **Step 4:** Any DFA state (subset) that contains at least one NFA final state is itself designated as a final state of the DFA.

Applying the Algorithm: Let $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{0, 1\}$, NFA transitions: $\delta(q_0, 0) = \{q_0\}$, $\delta(q_0, 1) = \{q_1\}$, $\delta(q_1, 0) = \{q_2\}$, $\delta(q_1, 1) = \{q_1\}$, $\delta(q_2, 0) = \{q_1\}$, $\delta(q_2, 1) = \{q_2\}$, with final state $\{q_2\}$.

DFA State (subset)	On 0	On 1
$A = \{q_0\}$ (start)	$\{q_0\} = A$	$\{q_1\} = B$
$B = \{q_1\}$	$\{q_2\} = C$	$\{q_1\} = B$
$C = \{q_2\}$ (final)	$\{q_1\} = B$	$\{q_2\} = C$

Resulting DFA: $Q' = \{A, B, C\}$, start state A , final state $F' = \{C\}$, and the transition table above defines δ' completely. Since every subset produced already appears among A, B, C , the algorithm terminates after processing exactly three DFA states, giving a minimal-looking DFA directly.

Fig — Equivalent DFA after subset construction

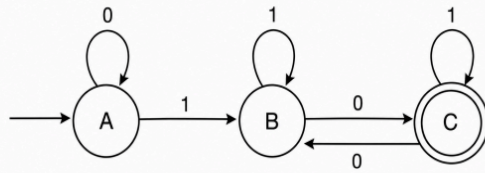
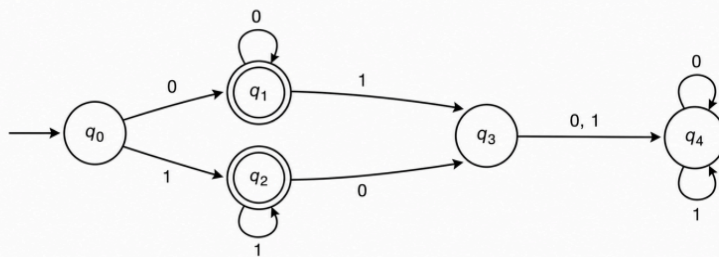


Fig — Equivalent DFA after subset construction

Verification with Test String: For input '0101', starting at A: $A \xrightarrow{(0)} A \xrightarrow{(1)} B \xrightarrow{(0)} C \xrightarrow{(1)} C$, ending in C which is final, so '0101' is accepted by both the NFA and the constructed DFA, confirming the conversion preserves the accepted language exactly.

Q9 Explain the algorithm for minimization of a DFA. Apply it to minimize the given DFA. [8M]

Fig — Given DFA to be minimized



Ans

DFA minimization reduces a given DFA to an equivalent DFA with the smallest possible number of states, while accepting exactly the same language. Two states can be merged into one if no input string can ever distinguish between them — that is, if starting from either state and reading the same string always leads either to both being final or both being non-final.

Algorithm — Equivalence (Partitioning) Method: The standard algorithm proceeds by successively refining a partition of the states.

- **Step 1:** Remove unreachable states — states that cannot be reached from the start state by any input string — since they play no role in the language accepted.
- **Step 2:** Create the initial partition with exactly two groups: the set of final states F , and the set of non-final states $Q - F$, since these two groups are trivially distinguishable (one accepts ϵ -continuation, the other does not).
- **Step 3:** For each group in the current partition, check whether all states in that group go to the same group on every input symbol. If some states in a group go to a different group than others on some symbol, split that group into sub-groups accordingly.
- **Step 4:** Repeat Step 3 until the partition stops changing (a fixed point is reached) — this final partition gives the equivalence classes of the minimized DFA.

- **Step 5:** Merge all states of each equivalence class into a single state of the minimized DFA, and redraw the transition table using these merged states.

Applying the Algorithm: Using the given DFA with $Q = \{q_0, q_1, q_2, q_3, q_4\}$, $F = \{q_1, q_2\}$, initial partition $P_0 = \{ \{q_1, q_2\}, \{q_0, q_3, q_4\} \}$.

Group	On 0	On 1
$\{q_1, q_2\}$	$q_1 \rightarrow q_1$ (stays in $\{q_1, q_2\}$); $q_2 \rightarrow q_3$ (goes to $\{q_0, q_3, q_4\}$) — DIFFERENT, so split	$q_1 \rightarrow q_3$ (goes out); $q_2 \rightarrow q_2$ (stays) — confirms split
$\{q_0, q_3, q_4\}$	$q_0 \rightarrow q_1$ (out to $\{q_1, q_2\}$); $q_3 \rightarrow q_4$ (stays); $q_4 \rightarrow q_4$ (stays) — split needed	$q_0 \rightarrow q_2$ (out); $q_3 \rightarrow q_4$ (stays); $q_4 \rightarrow q_4$ (stays)

Since $\{q_1, q_2\}$ splits (q_1 stays within the group on 0, q_2 leaves), and $\{q_0, q_3, q_4\}$ also splits (q_0 leaves, q_3 and q_4 stay together and behave identically), the refined partition becomes $P_1 = \{ \{q_1\}, \{q_2\}, \{q_0\}, \{q_3, q_4\} \}$. Checking further, q_3 and q_4 both self-loop to $\{q_3, q_4\}$ on every symbol, and no state distinguishes them any further, so the partition stabilises at P_1 .

Minimized State	On 0	On 1	Final?
$A = \{q_0\}$	B	C	No
$B = \{q_1\}$	B	D	Yes
$C = \{q_2\}$	D	C	Yes
$D = \{q_3, q_4\}$	D	D	No

Fig — Minimized DFA after merging equivalent states

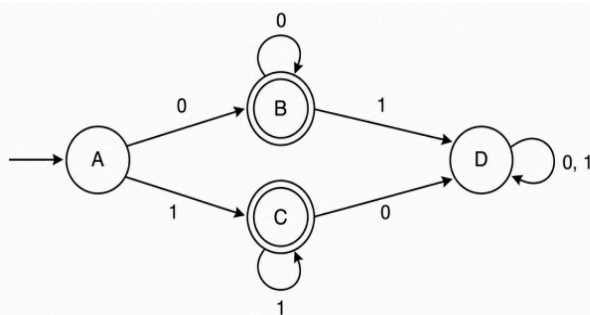


Fig — Minimized DFA after merging equivalent states

Result: The original 5-state DFA is reduced to an equivalent 4-state minimized DFA, since states q_3 and q_4 were found to be indistinguishable and were merged into the single state D, while q_0 , q_1 and q_2 remained as separate distinguishable states.

Q10	Design a DFA over alphabet $\Sigma = \{a, b\}$ that accepts strings which (i) contain exactly 3 occurrences of the symbol 'a' and (ii) contain at least 3 occurrences of the symbol 'a'. [8M] Ans Counting the number of occurrences of a particular symbol is one of the most common categories of DFA design questions, and is solved by using states to represent the running count of 'a' symbols seen so far, saturating the count once the required threshold is crossed. Part (i) — Exactly 3 occurrences of 'a': Since the count of a's must be tracked exactly up to 3 and then must not exceed 3, four 'counting' states are required for counts 0,1,2,3, plus one trap state to absorb any string with more than 3 a's. • States: $Q = \{q_0, q_1, q_2, q_3, q_{\text{trap}}\}$, where q_i represents that exactly i occurrences of 'a' have been seen so far, for $i = 0, 1, 2, 3$. • Start state: q_0, since zero a's have been seen before reading any input. • Final state: $F = \{q_3\}$, because the string must end with the count frozen at exactly 3. • Transitions: $\delta(q_i, 'a') = q_{i+1}$ for $i=0,1,2$; $\delta(q_3, 'a') = q_{\text{trap}}$ (a fourth a is not allowed); $\delta(q_i, 'b') = q_i$ for every state including q_{trap} (symbol b never changes the a-count); $\delta(q_{\text{trap}}, \text{any symbol}) = q_{\text{trap}}$. Fig — DFA for exactly 3 occurrences of a <pre> graph LR start(()) --> q0((q0)) q0 -- b --> q0 q0 -- a --> q1((q1)) q1 -- b --> q1 q1 -- a --> q2((q2)) q2 -- b --> q2 q2 -- a --> q3(((q3))) q3 -- b --> q3 q3 -- a --> q_trap(((q_trap))) q_trap -- b --> q_trap q_trap -- a --> q_trap style start fill:none,stroke:none </pre> Fig — DFA for exactly 3 occurrences of a Part (ii) — At least 3 occurrences of 'a': Here the only change needed from part (i) is that once the count reaches 3, the machine must remain accepting for any further a's or b's, since 3 or more a's already satisfies the 'at least' condition. • States: $Q = \{q_0, q_1, q_2, q_3\}$, where q_3 now represents 'three or more a's seen' instead of 'exactly three'. • Final state: $F = \{q_3\}$, and no trap state is required because q_3 simply self-loops on further a's instead of moving to a dead state. • Transitions: $\delta(q_i, 'a') = q_{i+1}$ for $i=0,1,2$; $\delta(q_3, 'a') = q_3$ (self-loop, since count only needs to stay ≥ 3); $\delta(q_i, 'b') = q_i$ for all $i=0,1,2,3$.
-------------------	---

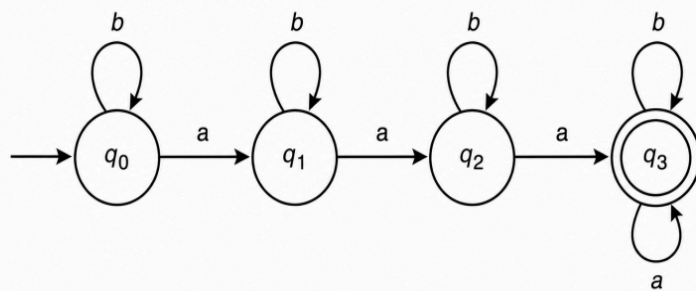
Fig — DFA for at least 3 occurrences of a

Fig — DFA for at least 3 occurrences of a

Verification: For part (i), input 'aabab' has exactly 3 a's: $q_0 \rightarrow (a)q_1 \rightarrow (a)q_2 \rightarrow (b)q_2 \rightarrow (a)q_3 \rightarrow (b)q_3$, ending in q_3 , accepted. Input 'aaaab' has 4 a's: after the fourth a the machine enters q_{trap} , and stays there, so it is correctly rejected by the exactly-3 DFA. For part (ii), the same input 'aaaab' would end in q_3 (self-loop absorbing the extra a), and is correctly accepted since it has at least 3 a's.

The only structural difference between an 'exactly n' counting DFA and an 'at least n' counting DFA is therefore whether the final counting state transitions out to a trap state or loops back onto itself on the counted symbol — a distinction that is very frequently tested and easy to get wrong under exam pressure.

Q11

Design a DFA which accepts a decimal number divisible by 3. Explain the logic used in the construction. [7M]

Ans

A DFA that decides divisibility by 3 is a classic example of using automaton states to represent the residues (remainders) of a running arithmetic computation, rather than counting occurrences of a symbol.

Logic Used: When a decimal number is built digit by digit from left to right, its remainder modulo 3 can be updated incrementally: if the current number has remainder r when divided by 3, and the next digit d is appended, then the new number equals $(10 \times \text{old_number} + d)$, and its remainder modulo 3 is $(10r + d) \bmod 3$, which simplifies to $(r + d) \bmod 3$ since $10 \bmod 3 = 1$. Each of the three possible remainders 0, 1, 2 is represented by exactly one state, and reading a digit simply moves the machine to the state corresponding to the new remainder.

- **States:** $Q = \{q_0, q_1, q_2\}$, where state q_r represents that the decimal number read so far leaves remainder r when divided by 3.
- **Alphabet:** $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$, the ten decimal digits.
- **Start state:** q_0 , since the empty prefix read so far corresponds to the number 0, which has remainder 0.
- **Final state:** $F = \{q_0\}$, because a number is divisible by 3 exactly when its remainder is 0.

- **Transition rule:** $\delta(qr, d) = q((r+d) \bmod 3)$ for every digit d and every state qr — this single rule defines all 30 transitions of the table compactly.

Formula Name: Remainder Update Rule

Expression: $\text{new_remainder} = (\text{old_remainder} + \text{digit}) \bmod 3$

Where:

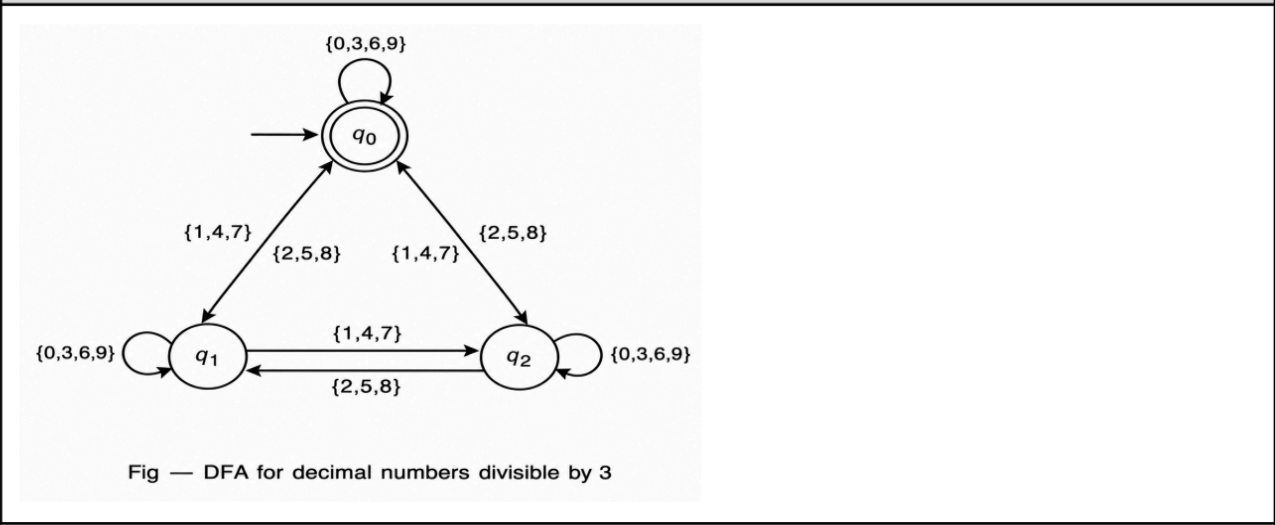
old_remainder = the state before reading the digit

digit = the decimal digit just read (0–9)

Condition: Derived from $10 \equiv 1 \pmod{3}$, so appending a digit is equivalent to adding it to the existing remainder

State (remainder)	On digits giving +0 mod3 (0,3,6,9)	Behaviour on other digit classes
q0	stays at q0	digits $\equiv 1 \pmod{3}$ (1,4,7) $\rightarrow$ q1; digits $\equiv 2 \pmod{3}$ (2,5,8) $\rightarrow$ q2
q1	goes to q1 (adds 0)	digits $\equiv 1 \pmod{3} \rightarrow$ q2; digits $\equiv 2 \pmod{3} \rightarrow$ q0
q2	goes to q2 (adds 0)	digits $\equiv 1 \pmod{3} \rightarrow$ q0; digits $\equiv 2 \pmod{3} \rightarrow$ q1

Fig — DFA for decimal numbers divisible by 3



Verification: Consider input '123'. Starting at q_0 : digit 1 ($\equiv 1 \pmod{3}$) $\rightarrow$ q_1 ; digit 2 ($\equiv 2 \pmod{3}$) $\rightarrow$ q_0 ; digit 3 ($\equiv 0 \pmod{3}$) $\rightarrow$ q_0 . The machine ends in q_0 , which is final, and indeed $123 = 3 \times 41$ is divisible by 3. Consider input '124': $q_0 \rightarrow (1)q_1 \rightarrow (2)q_0 \rightarrow (4, \equiv 1 \pmod{3})q_1$, ending in q_1 , not final, and indeed 124 is not divisible by 3, confirming the design is correct.

The same remainder-tracking technique generalises directly to checking divisibility by any constant k in any given base b , by using k states and computing the transition rule as $(r \times b + d) \bmod k$, which is a standard extension frequently asked in exams for binary divisibility by 3.

Q12

Define Moore machine. Explain its construction with a suitable transition table and diagram. [6M]

Ans

A Moore machine is a finite automaton with output, in which the output produced depends solely on the current state of the machine and not on the input symbol currently being read.

Moore Machine — Formal Definition: A Moore machine is defined as a 6-tuple $M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where Q is a finite set of states, Σ is the input alphabet, Δ is the output alphabet, $\delta: Q \times \Sigma \rightarrow Q$ is the state transition function, $\lambda: Q \rightarrow \Delta$ is the output function that associates an output symbol with every state, and q_0 is the start state. Because λ depends only on Q , every state of a Moore machine has one single, fixed output symbol attached to it, which is emitted whenever the machine is in that state.

Construction Method: To design a Moore machine for a given specification, states are first identified to represent the relevant 'memory' needed of the input read so far (for example, which suffix of the input has just been seen), each state is labelled with the output it must produce, and transitions are added for every input symbol from every state according to the required behaviour.

Illustrative Example — 1's Complement of a Binary Number: A Moore machine that outputs the 1's complement of an input binary string needs only two states, since the output for each input bit depends only on the bit read, which is trivially state-independent here but still modelled with a state per bit for illustration.

Present State	Output (λ)	Next State on 0	Next State on 1
q_0 (start)	1	q_1	q_0
q_1	0	q_1	q_0

Fig — Moore machine for 1's complement of a binary number

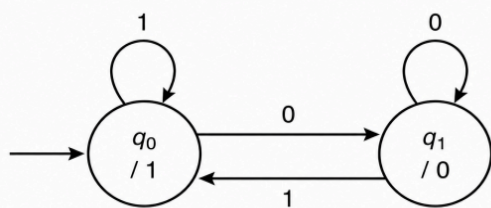


Fig — Moore machine for 1's complement of a binary number

Key Property — Output Length: In a Moore machine, the output string produced is always exactly one symbol longer than the input string, because the machine emits the output of the start state before reading any input symbol at all, and then emits one more output symbol for every state entered thereafter.

Applications: Moore machines are widely used to design sequence detectors, vending machine controllers, traffic-light controllers, and digital counters, wherever the output naturally corresponds to 'being in a particular condition' rather than to a specific transition event.

Because the output depends purely on the state, a Moore machine is generally considered slightly easier to reason about and to implement in synchronous digital hardware, since the output can be latched directly from the state register without additional combinational logic on the input lines.

Q13

Define Mealy machine. Explain its construction with a suitable transition table and diagram. [6M]

Ans

A Mealy machine is a finite automaton with output in which the output produced at each step depends on both the current state and the input symbol currently being read, rather than on the state alone.

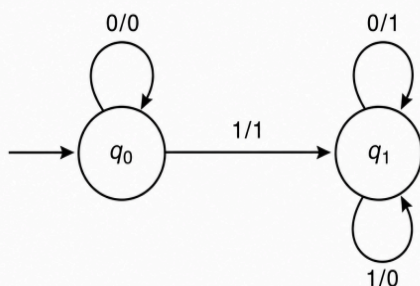
Mealy Machine — Formal Definition: A Mealy machine is defined as a 6-tuple $M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where Q is a finite set of states, Σ is the input alphabet, Δ is the output alphabet, $\delta: Q \times \Sigma \rightarrow Q$ is the state transition function, $\lambda: Q \times \Sigma \rightarrow \Delta$ is the output function, and q_0 is the start state. Because λ takes both a state and an input symbol as arguments, the output is associated with each transition (edge) of the machine rather than with each state (circle).

Construction Method: To construct a Mealy machine, states are identified to capture the relevant memory of the input processed so far, and every transition arrow in the diagram is labelled with both the input symbol that triggers it and the output symbol produced on that transition, typically written as 'input/output' on the edge label.

Illustrative Example — 2's Complement of a Binary Number (processed right to left conceptually, modelled left to right with state memory): A Mealy machine to compute the 2's complement keeps track of whether the least-significant 1 has already been seen, since bits before the first 1 are copied unchanged and bits after are complemented.

Present State	Input 0 → Output/Next	Input 1 → Output/Next
q0 (start, before first 1)	output 0 / stay q0	output 1 / move to q1
q1 (after first 1 seen)	output 1 / stay q1	output 0 / stay q1

Fig — Mealy machine for 2's complement of a binary number



Key Property — Output Length: In a Mealy machine, the output string produced has exactly the same length as the input string, since one output symbol is emitted precisely for every input symbol consumed, unlike a Moore machine which produces one extra initial output symbol.

Applications: Mealy machines are commonly used for control-unit design in processors, serial adders, edge-triggered digital circuits, and situations where the output must react immediately and precisely to a specific input transition rather than reflecting a static condition.

Because output depends on the input symbol as well as the state, a Mealy machine can react to an input change within the same clock cycle without waiting for a state update, which typically produces a machine with fewer states than an equivalent Moore machine solving the same problem — a trade-off frequently asked about in comparison questions.

Q14	Differentiate between Moore machine and Mealy machine (any six points) with suitable examples. [6M]			
Ans	Moore and Mealy machines are the two standard models of finite automata with output, and while every Mealy machine can be converted to an equivalent Moore machine and vice versa, they differ significantly in structure, output timing, and typical circuit realisation.			
	Sr. No.	Parameter	Moore Machine	Mealy Machine
1	Output depends on	Only the current state, $\lambda: Q \rightarrow \Delta$	Current state and input symbol, $\lambda: Q \times \Sigma \rightarrow \Delta$	
2	Output associated with	Each state (node) of the diagram	Each transition (edge) of the diagram	
3	Output string length	One symbol longer than the input string (extra output for start state)	Exactly equal to the length of the input string	
4	Response time to input change	Output changes only after the next state is entered (one step delay)	Output changes immediately as the input is applied, within the same transition	
5	Number of states typically needed	Usually needs more states for the same language/behaviour	Usually needs fewer states for the same behaviour	
6	Circuit / hardware realisation	Output logic depends only on present-state flip-flops, generally more stable / glitch-free	Output logic depends on both present-state flip-flops and current input lines, can glitch briefly on input change	
7	Ease of design from specification	Slightly easier to design directly, since output is tied to a clearly defined condition	Slightly more compact but requires careful tracking of input/output pairs on each edge	
8	Conversion	Can always be converted into an equivalent Mealy machine	Can always be converted into an equivalent Moore machine (sometimes requiring extra states)	

Example Contrast: For a machine detecting the substring '01' and producing output 1 upon detection, a Moore machine needs at least three states — 'no progress', 'seen 0', and 'seen 01' with output 1 attached to the last state — while an equivalent Mealy machine can often achieve the same detection using only two states, since the output '1' can be attached directly to the transition edge that consumes the second symbol '1' rather than requiring a separate output-bearing state.

Conversion Note: When converting a Mealy machine to a Moore machine, each state of the Mealy machine may need to be split into multiple Moore states, one for every distinct output value that could arrive at that state along different incoming edges, which is why the resulting Moore machine can have more states than the original Mealy machine. Conversely, converting a Moore machine to a Mealy machine simply relabels the output of the destination state onto the incoming transition edge, so the number of states never increases in this direction.

In practical digital design, Mealy machines are often preferred when a minimal state count and immediate response are priorities, such as in fast serial communication protocols, whereas Moore machines are preferred when stable, glitch-free outputs synchronised to a clock are more important, such as in traffic-light or vending-machine controllers.

Q15

Design a Mealy machine that accepts strings ending in '00' or '11'. Convert the Mealy machine into its equivalent Moore machine. [9M]

Ans

This problem requires first identifying the states needed to remember the last one or two input symbols read, since detecting a specific suffix ('00' or '11') at the end of the string is inherently a 'memory of recent history' problem, and then producing an output of 1 exactly when that suffix has just been completed.

Design of the Mealy Machine: Four states are used: q0 the start state representing 'no relevant progress', q1 representing 'last symbol read was 0', q2 representing 'last symbol read was 1', and outputs are attached directly to the transition edges.

Present State	Input 0 → Output/Next	Input 1 → Output/Next
q0 (start)	0 / q1	0 / q2
q1 (last symbol 0)	1 / q1 (second 0 in a row → '00' detected)	0 / q2
q2 (last symbol 1)	0 / q1	1 / q2 (second 1 in a row → '11' detected)

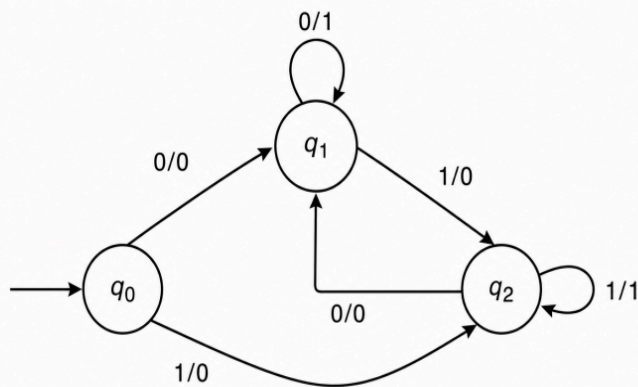
Fig — Mealy machine detecting strings ending in 00 or 11

Fig — Mealy machine detecting strings ending in 00 or 11

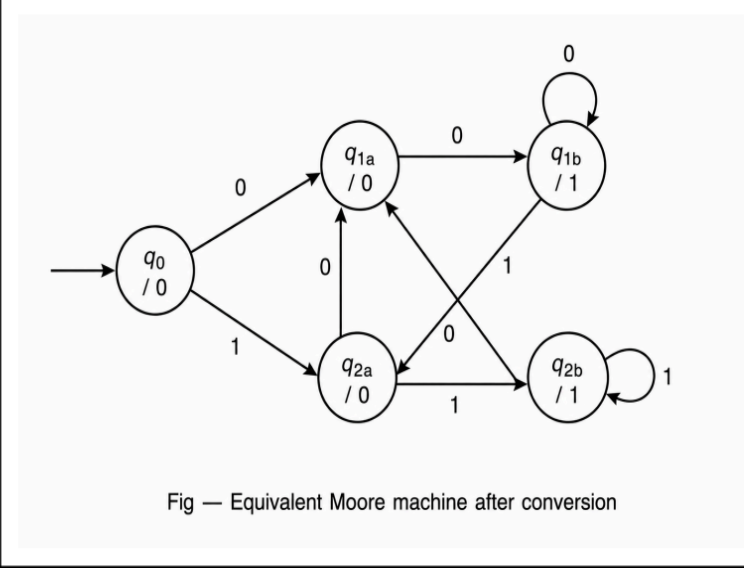
Verification of Mealy Machine: For input '1100', processing symbol by symbol: $q_0 \rightarrow (1/0) q_2 \rightarrow (1/1) q_2 \rightarrow (0/0) q_1 \rightarrow (0/1) q_1$. The output string is '0101', and the final output symbol is 1, correctly signalling that the string ends in '00'. For input '101', $q_0 \rightarrow (1/0) q_2 \rightarrow (0/0) q_1 \rightarrow (1/0) q_2$, giving output '000' with final output 0, correctly showing the string neither ends in '00' nor '11'.

Conversion of Mealy Machine to Equivalent Moore Machine: To convert, every state of the Mealy machine is examined for the distinct output values that arrive at it along different incoming edges; wherever more than one distinct output value arrives at the same state, that state must be split into separate copies, one for each output value, so the output can be attached to the state itself as required by the Moore model.

- **State q_0 :** Is the start state with no incoming edges from within the machine in this design, so it is kept as a single state q_0 with output 0 (arbitrary/default, since it is never an accepting configuration by itself).
- **State q_1 :** Receives output 0 from q_0 (on input 0) and output 0 from q_2 (on input 0), but also receives output 1 from itself (self-loop on input 0). Since outputs 0 and 1 both arrive at q_1 , it is split into q_{1a} with output 0 and q_{1b} with output 1.
- **State q_2 :** Similarly receives output 0 (from q_0 on 1, and from q_1 on 1) and output 1 (from itself on input 1), so it is split into q_{2a} with output 0 and q_{2b} with output 1.
-

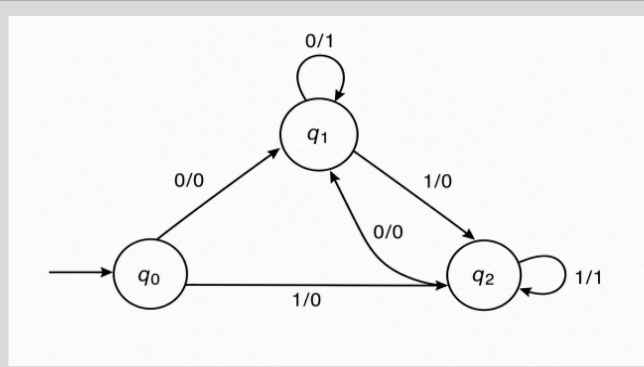
Moore State	Output λ	On 0	On 1
q_0 (start)	0	q_{1a}	q_{2a}
q_{1a}	0	q_{1b}	q_{2a}
q_{1b}	1	q_{1b}	q_{2a}
q_{2a}	0	q_{1a}	q_{2b}
q_{2b}	1	q_{1a}	q_{2b}

Fig — Equivalent Moore machine after conversion



Notice the Moore machine required five states (splitting q1 and q2 into two output-variants each) compared to the Mealy machine's three states, which is a direct illustration of the general rule that converting Mealy to Moore can increase the state count, while the language accepted and the pattern of outputs (apart from the harmless extra leading output symbol) remain equivalent.

Consider the following Mealy machine. Construct the equivalent Moore machine, and also differentiate between Moore and Mealy machines (any four points). [7M]



Q16

The given transition diagram is first read off into a transition table showing, for every state and every input symbol, the output produced and the next state reached, since this tabular form makes the state-splitting procedure needed for Moore conversion much easier to apply systematically.

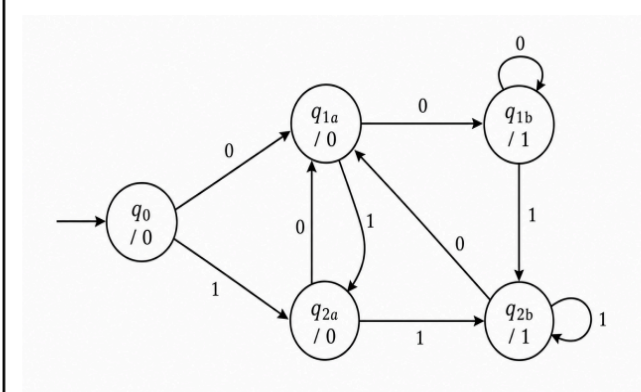
Present State	Input 0 → Output/Next	Input 1 → Output/Next
q0 (start)	0 / q1	0 / q2
q1	1 / q1	0 / q2
q2	0 / q1	1 / q2

Step 1 — Identify Distinct Incoming Outputs per State: For q_0 , there are no incoming edges from within the machine, so it keeps a single copy with a default output, say 0. For q_1 , incoming edges arrive from q_0 (output 0), from itself (output 1), and from q_2 (output 0); since both 0 and 1 arrive at q_1 , it must be split into two copies. For q_2 , incoming edges arrive from q_0 (output 0), from q_1 (output 0), and from itself (output 1); again both 0 and 1 arrive, so q_2 must also be split into two copies.

Step 2 — Build Split States: Let q_{1a} carry output 0 and q_{1b} carry output 1 (both representing 'q1' with different histories of output), and let q_{2a} carry output 0 and q_{2b} carry output 1 similarly for q_2 .

Moore State	Output λ	On 0	On 1
q_0 (start)	0	q_{1a}	q_{2a}
q_{1a}	0	q_{1b}	q_{2a}
q_{1b}	1	q_{1b}	q_{2a}
q_{2a}	0	q_{1a}	q_{2b}
q_{2b}	1	q_{1a}	q_{2b}

Fig — Equivalent Moore machine constructed from the given Mealy machine



Verification: Testing input '101' on the Mealy machine gives: $q_0 \rightarrow (1/0)q_2 \rightarrow (0/0)q_1 \rightarrow (1/0)q_2$, output string '000'. On the constructed Moore machine, starting output is 0 (from q_0), then $q_0 \rightarrow q_{2a}$ (output 0) $\rightarrow q_{1a}$ (output 0) $\rightarrow q_{2a}$ (output 0), giving output string '0000' — after dropping the extra leading symbol contributed by the start state, the outputs '000' match exactly, confirming correctness of the conversion.

Differentiation (four points): (1) Moore output depends only on state, Mealy output depends on state and input symbol. (2) Moore output string is one symbol longer than input, Mealy output string equals input length. (3) Moore machines generally need more states for the same behaviour, Mealy machines are usually more compact. (4) Moore output is stable and tied to the state register alone, while Mealy output can change immediately with the input and is tied to the transition logic.

Q17 Design a Moore machine such that for every substring that ends in 'bab' the machine gives output 1, over alphabet $\Sigma = \{a, b\}$. Further convert the same Moore machine into an equivalent Mealy machine. [8M]

Ans This is a sequence-detector design problem where the machine must remember how much of the pattern 'bab' has been matched so far as a suffix of the input read, and must output 1 exactly in the state reached immediately after the full pattern 'bab' has just been completed.

Design of the Moore Machine: Four states are used to represent progress in matching the pattern: q0 (no relevant suffix matched, or 'start'), q1 (suffix 'b' matched), q2 (suffix 'ba' matched), q3 (suffix 'bab' just completed, output 1). Care must be taken to correctly redirect the machine on a mismatch, since input symbols can still contribute to a different partial match of the pattern (this is the same principle used in the KMP failure-function idea, applied informally).

State	Output λ	Meaning (suffix matched)	On a	On b
q0 (start)	0	No part of 'bab' matched yet	q0	q1
q1	0	'b' matched	q2	q1
q2	0	'ba' matched	q0	q3
q3	1	'bab' just completed	q2	q1

The transitions on mismatch are chosen carefully: from q2 (having matched 'ba'), reading 'a' does not continue the pattern, but since the string now ends in 'a' with no useful prefix for 'bab', it goes back to q0. From q3 (just matched 'bab', ending in 'b'), reading 'a' means the last two symbols are 'ba', so it moves to q2 (matched 'ba'), and reading 'b' from q3 keeps a matched suffix of 'b', moving to q1.

Fig — Moore machine detecting substring bab (output 1)

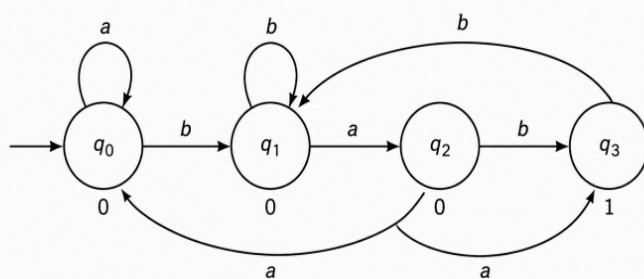


Fig. — Moore machine detecting substring bab (output 1)

Verification of Moore Machine: For input 'ababab', the state sequence is $q_0 \rightarrow (a)q_0 \rightarrow (b)q_1 \rightarrow (a)q_2 \rightarrow (b)q_3 \rightarrow (a)q_2 \rightarrow (b)q_3$, with outputs 0,0,0,0,1,0,1 (including the initial output of q0). The two output-1 positions correspond exactly to the two occurrences of 'bab' ending at position 4 and position 6 of 'ababab', confirming correct pattern detection.

Conversion to Equivalent Mealy Machine: Converting a Moore machine to a Mealy machine is simpler than the reverse direction: the output attached to a destination state in the Moore machine is

simply moved onto the incoming transition edge that leads into that state, and no state splitting or increase in the number of states is ever required.

Present State	Input a → Output/Next	Input b → Output/Next
q0 (start)	0 / q0 (output of destination q0)	0 / q1 (output of destination q1)
q1	0 / q2 (output of destination q2)	0 / q1 (output of destination q1)
q2	0 / q0 (output of destination q0)	1 / q3 (output of destination q3)
q3	0 / q2 (output of destination q2)	0 / q1 (output of destination q1)

Fig — Equivalent Mealy machine after conversion

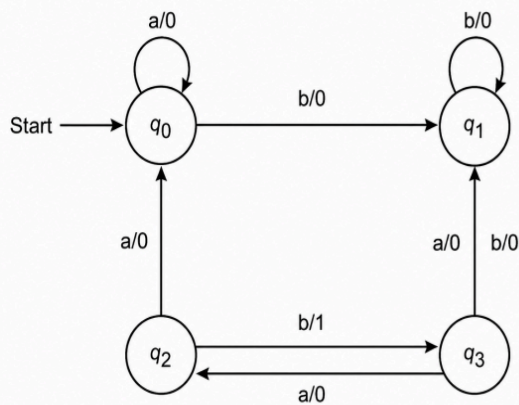


Fig — 4-State Mealy Machine

Observe that the number of states remains exactly four in both the Moore and Mealy versions, and only the position at which each output symbol is attached — to a state versus to an edge — has changed, illustrating why Moore-to-Mealy conversion never increases the state count while the reverse conversion often does.

Q18

What is the language accepted by a Finite Automaton? Define Regular Language with a suitable example. [5M]

Ans

Once a finite automaton is built, the most fundamental question that can be asked about it is precisely which set of input strings it accepts — this set is called the language of the automaton, and it links the machine model directly back to the notion of a formal language.

Language Accepted by an FA — Definition: For a finite automaton $M = (Q, \Sigma, \delta, q_0, F)$, the language accepted by M , written $L(M)$, is defined as the set of all strings w over Σ such that processing w starting from q_0 using the extended transition function δ ends in some state belonging to F . Formally, $L(M) = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}$. Any string not satisfying this condition is said to be rejected by M .